

Git and GitHub – A beginner's guide

What is Git?

Git is a Version Control System (VCS) used to track changes in files and projects over time.

It helps developers:

- Save project history
- Work collaboratively
- Restore previous versions
- Manage multiple features safely
- Track who made changes

Git is installed locally on your computer.

What is GitHub?

GitHub is a cloud-based platform where Git repositories are stored online.

GitHub helps developers:

- Store code online
- Collaborate with teams
- Share projects
- Review code
- Create pull requests
- Manage open-source projects

What is Version Control?

Version Control means tracking every change made to files.

Example Without Git

You may have files like:

- resume.docx

- resume_v2.docx
- resume_final.docx
- resume_final_latest.docx

This becomes confusing.

Example With Git

Git stores all changes in one project. You can:

- See change history
- Restore old versions
- Compare changes
- Work safely without creating duplicate files

Difference Between Git and GitHub

Git	GitHub
Version control tool	Cloud hosting platform
Installed locally	Works online
Tracks changes	Stores repositories
Command-line based	Web interface
Works offline	Requires internet

Key Git Concepts

1. Repository (Repo)

A repository is a project folder tracked by Git.

Example:

- Student-Portfolio
- AI-Projects
- Python-Practice

2. Working Directory

The place where you create and modify files.

Example: You edit:

- index.html
- app.py
- report.docx

These changes are in the working directory.

3. Staging Area

Temporary area before saving changes permanently.

Git uses staging to:

- Review changes
- Select files carefully
- Prepare commits

Command: **git add**

4. Commit

A commit is a saved checkpoint in project history.

Example:

```
git commit -m "Added login page"
```

5. Local Repository

Repository stored on your own computer.

6. Remote Repository

Repository stored online (GitHub).

7. Branch

A separate workspace for new features.

Branches help avoid damaging the main project.

8. Merge

Combining changes from one branch into another.

9. Pull Request (PR)

Request to merge code into another branch.

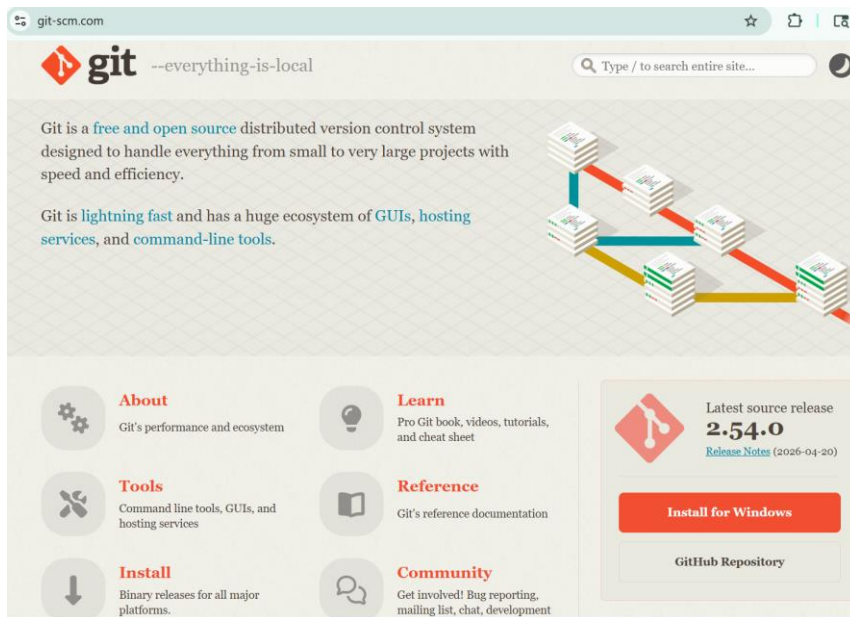
Used heavily in team projects.

Installing Git

Windows Installation

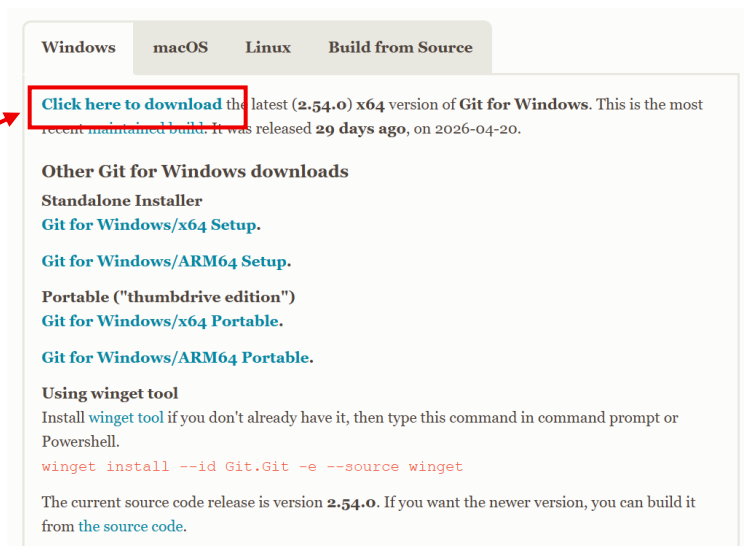
Step 1

Go to: <https://git-scm.com/>



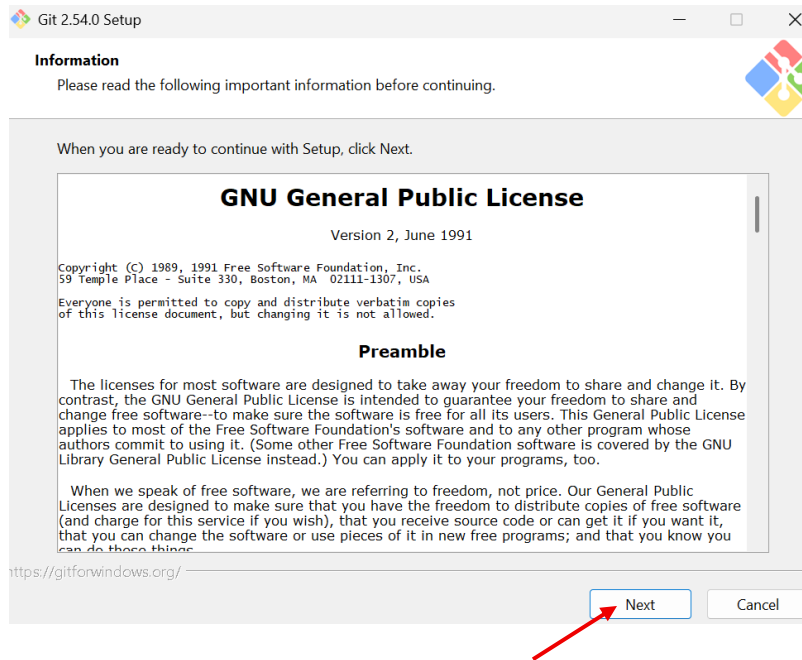
Step 2

Download Git for Windows.



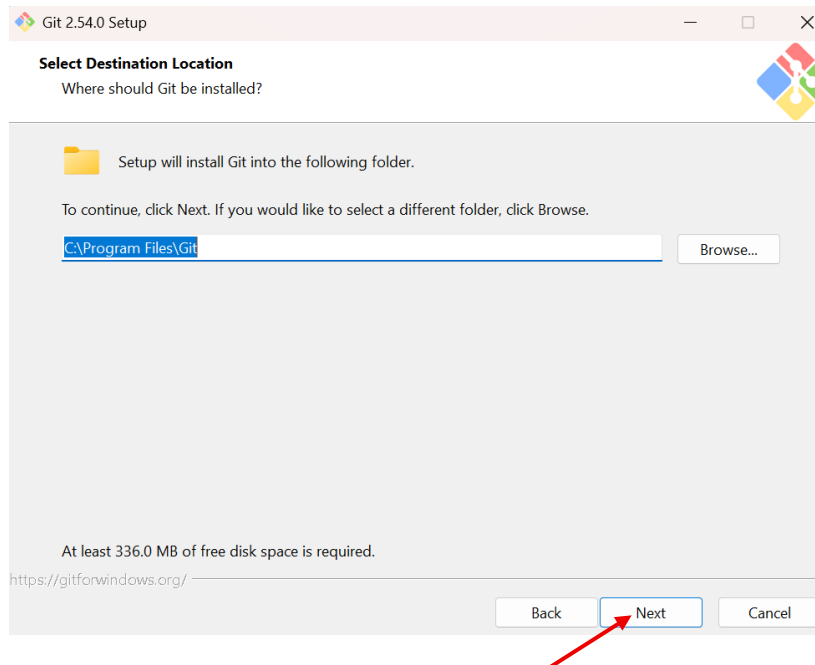
Step 3

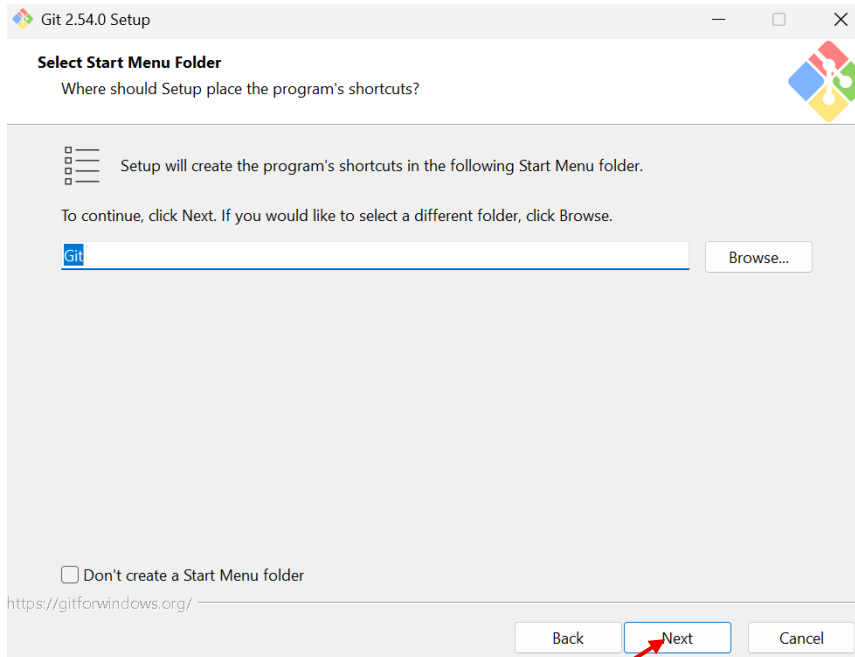
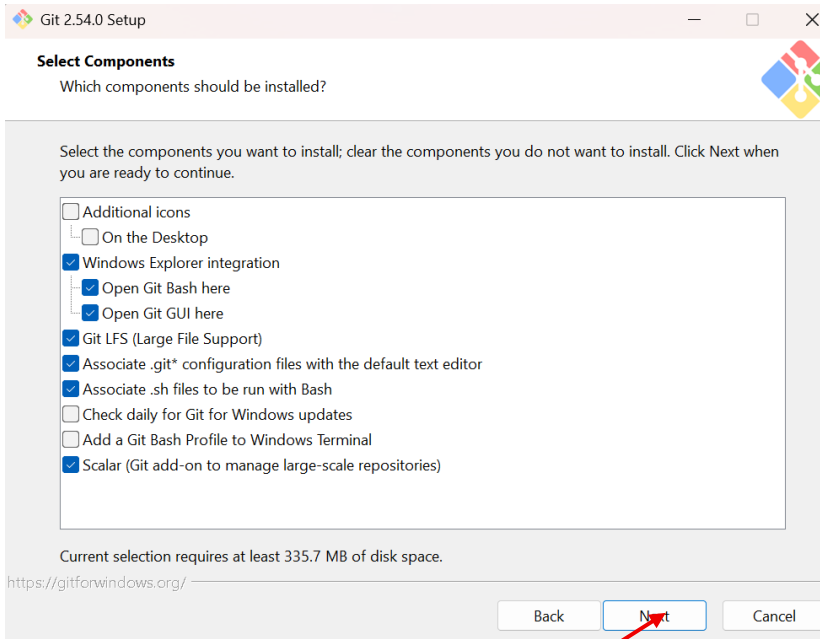
Run installer.

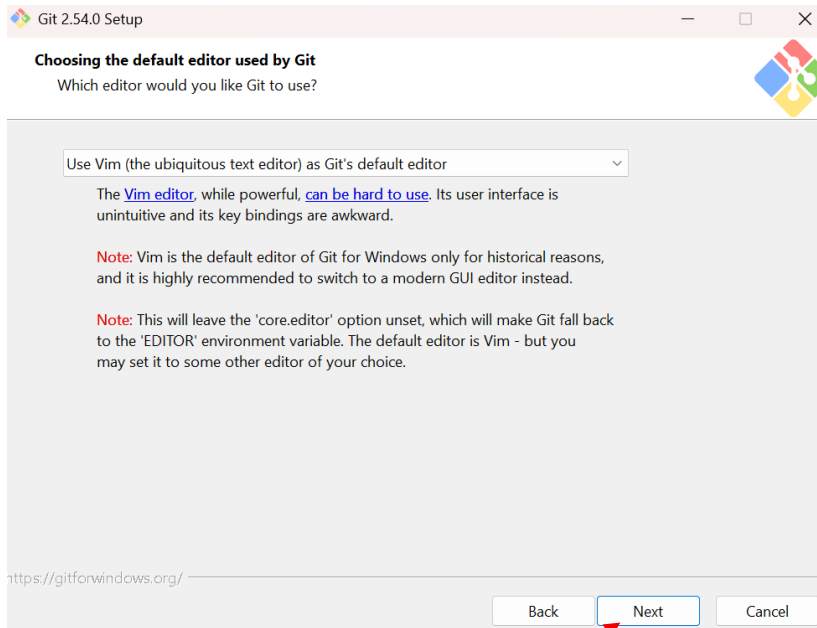


Step 4

Keep default settings.



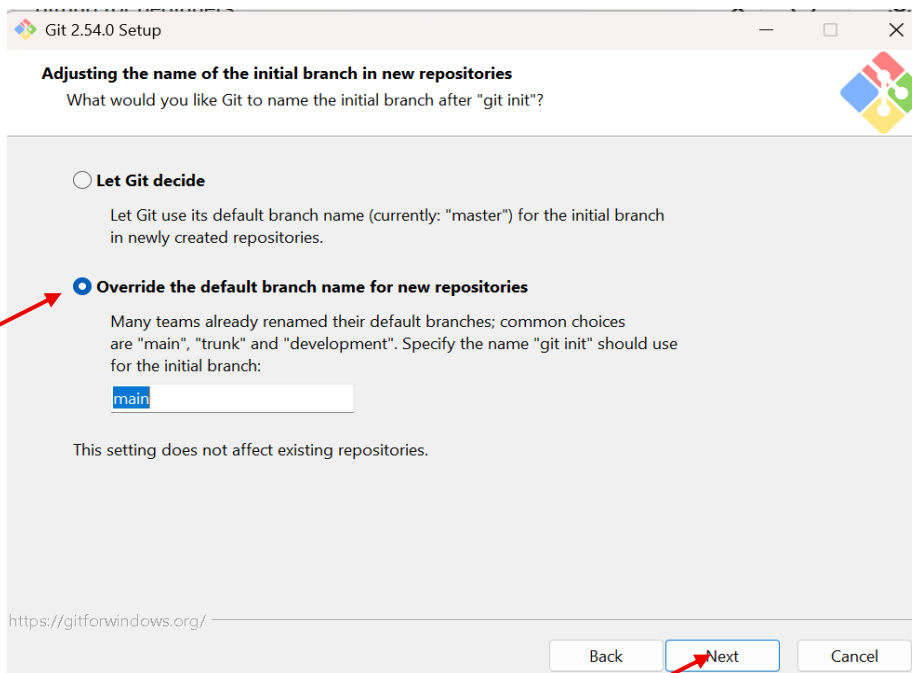


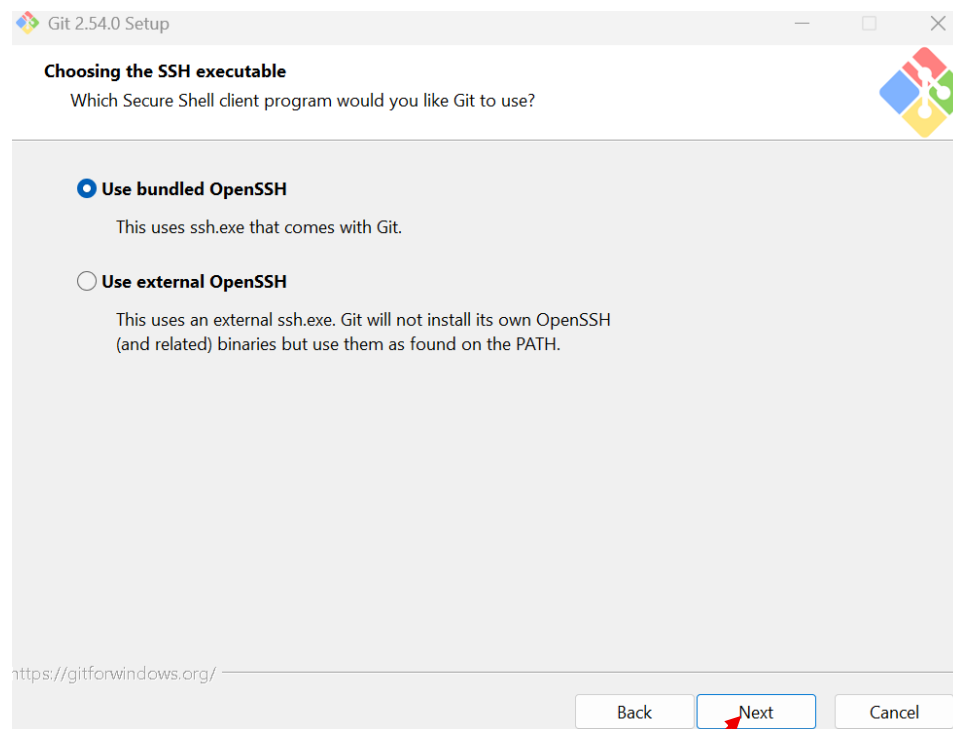
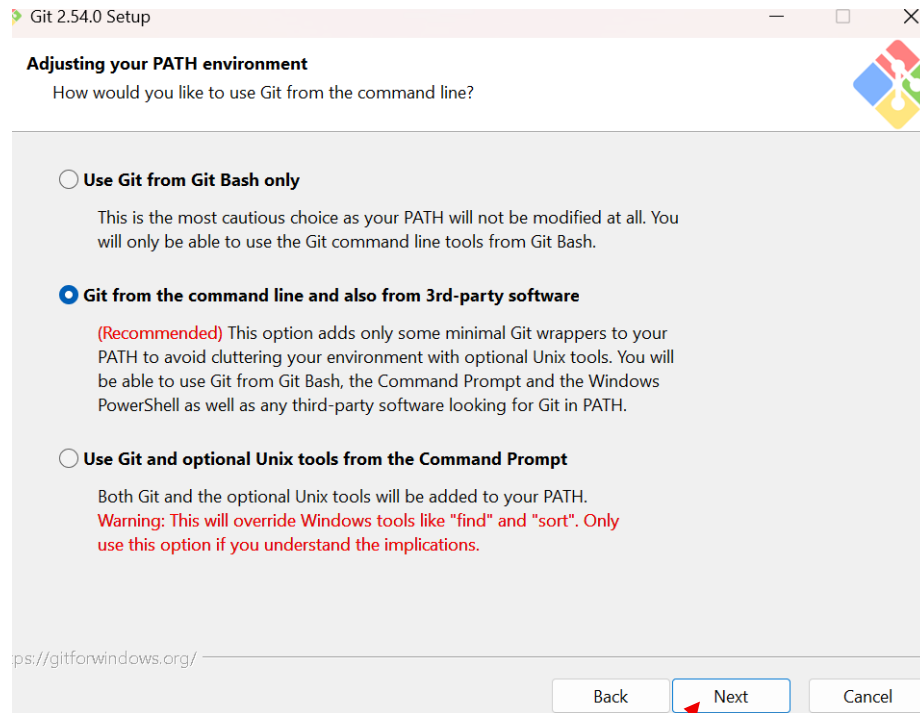


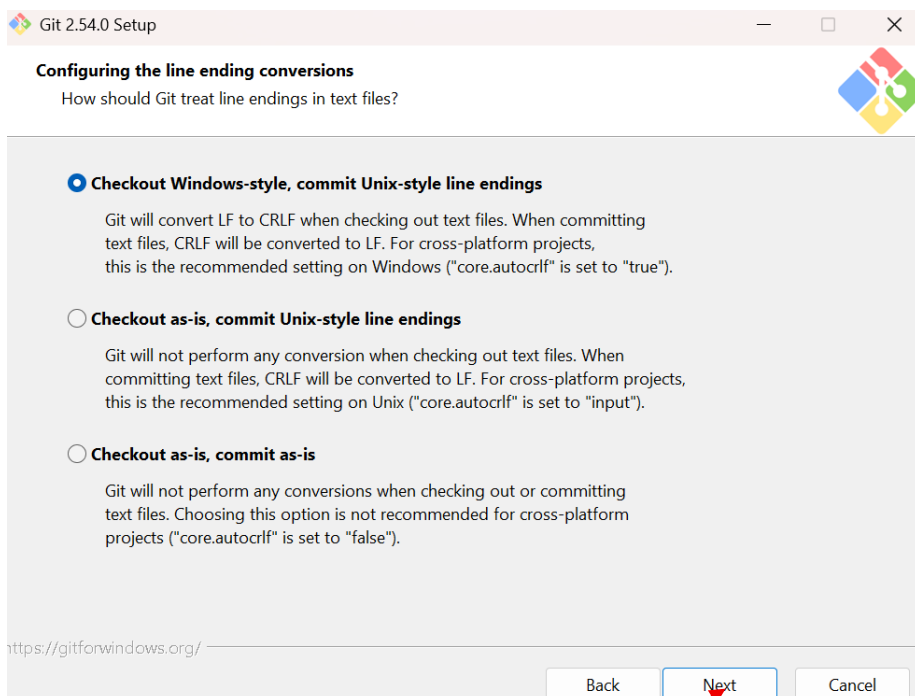
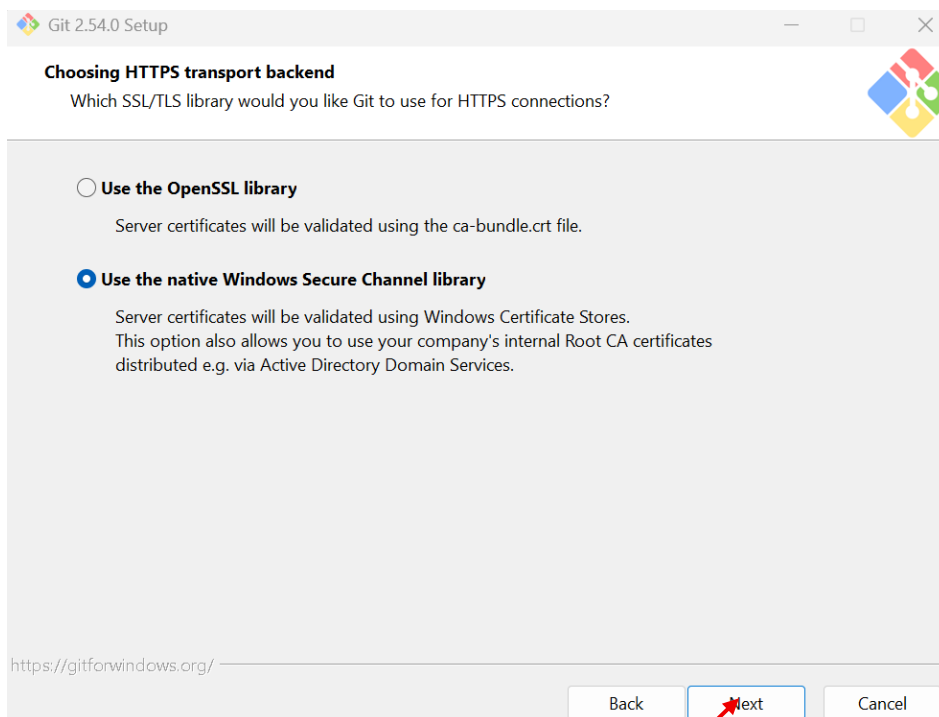
Step 5

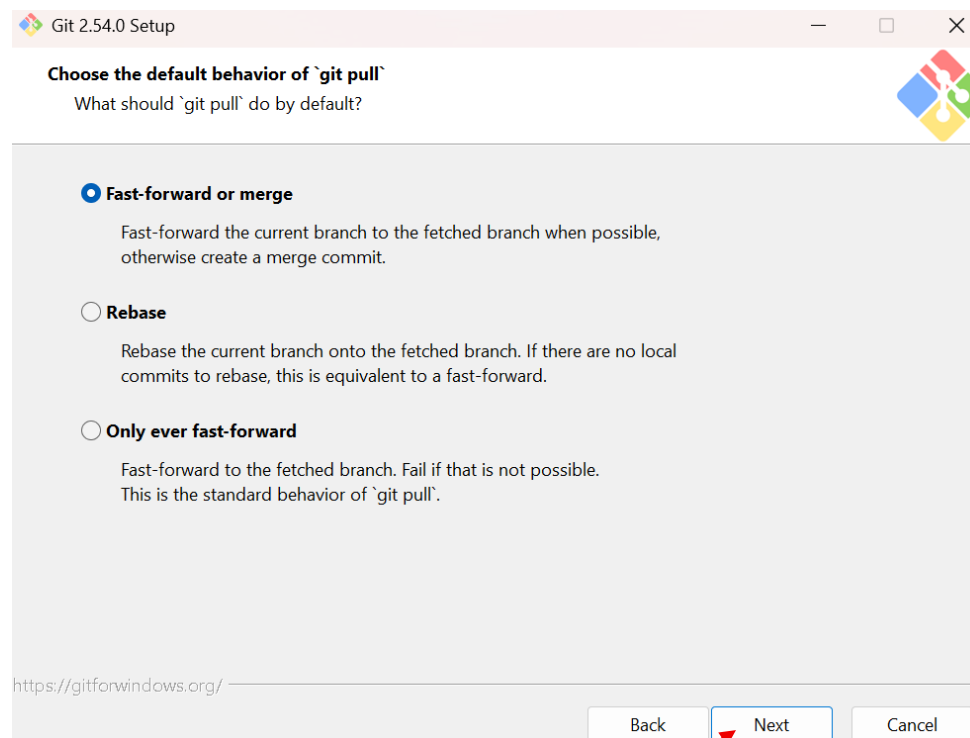
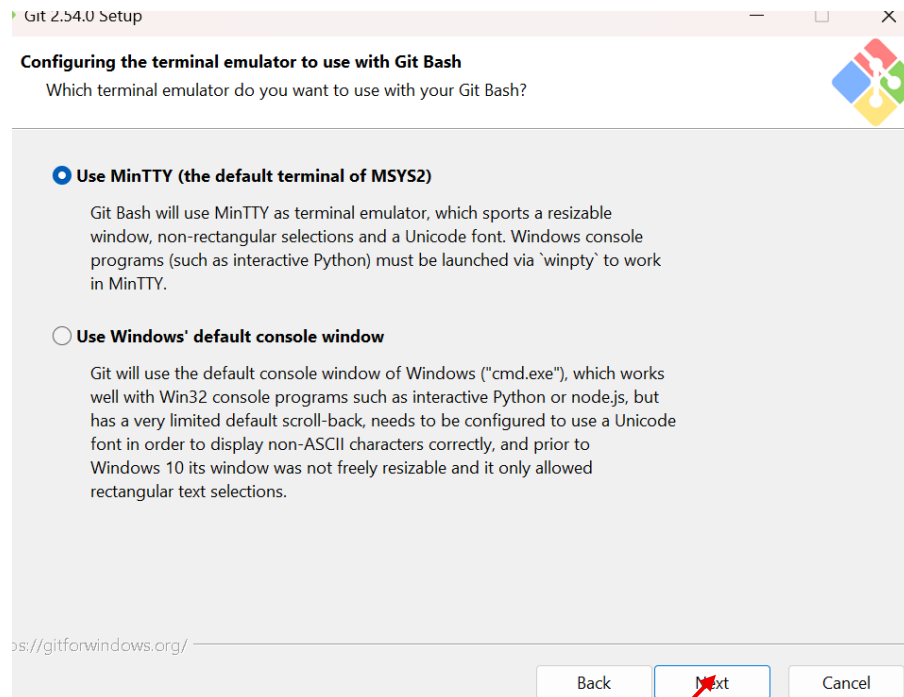
Set default branch name to:

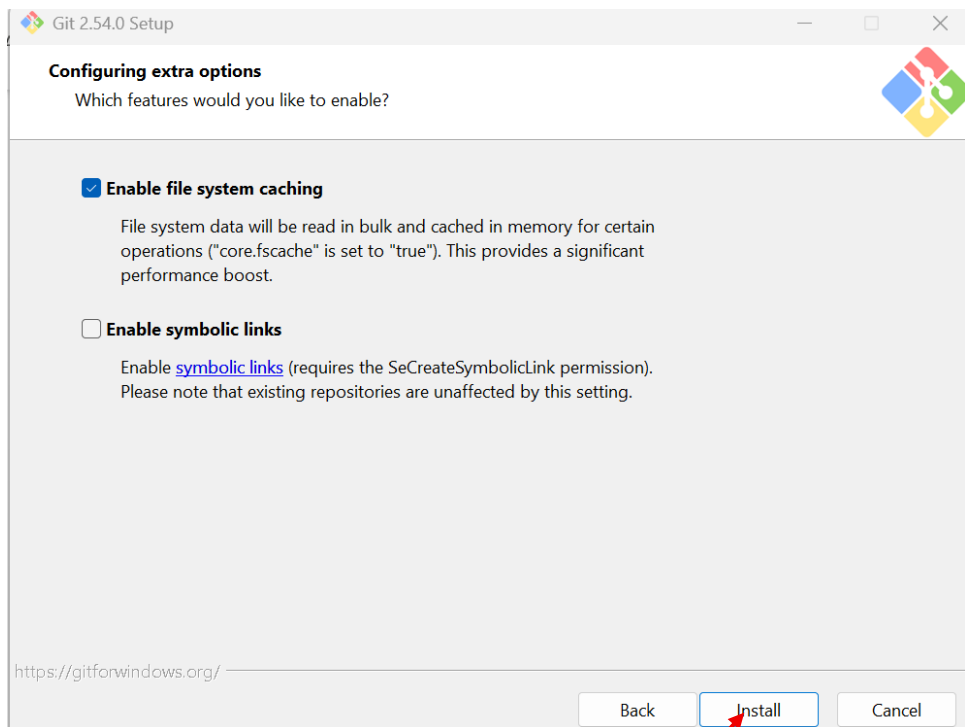
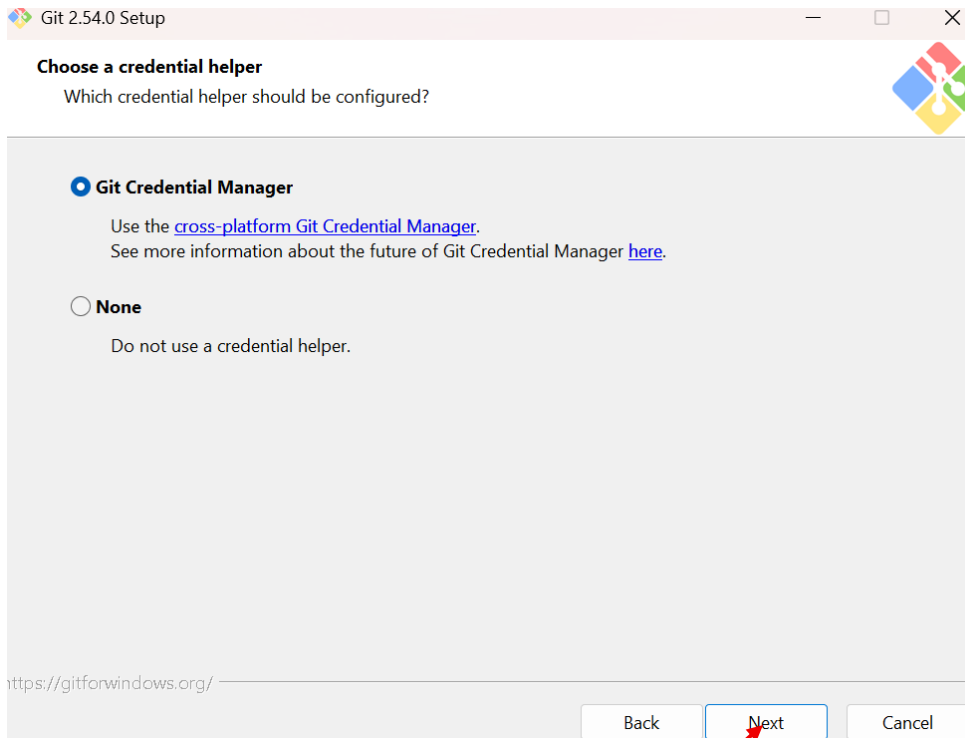
main





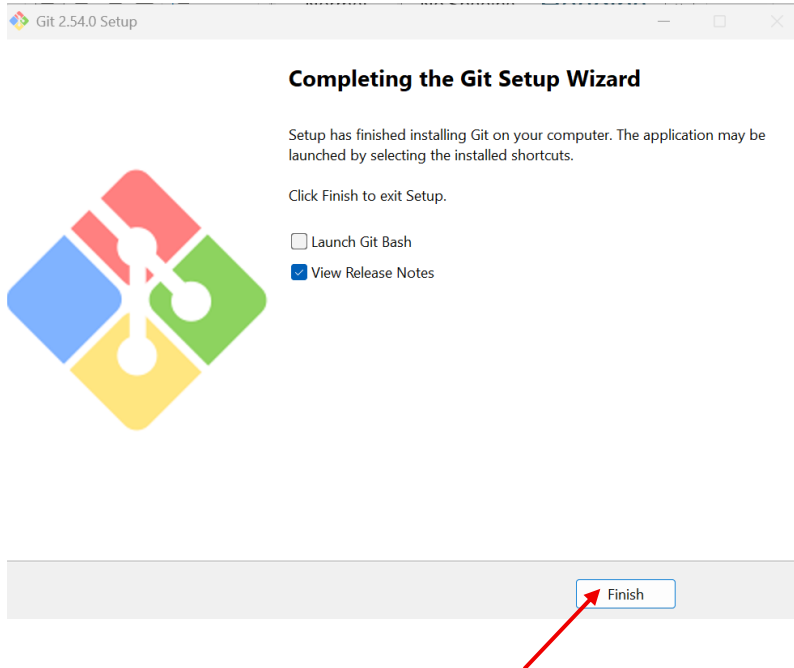






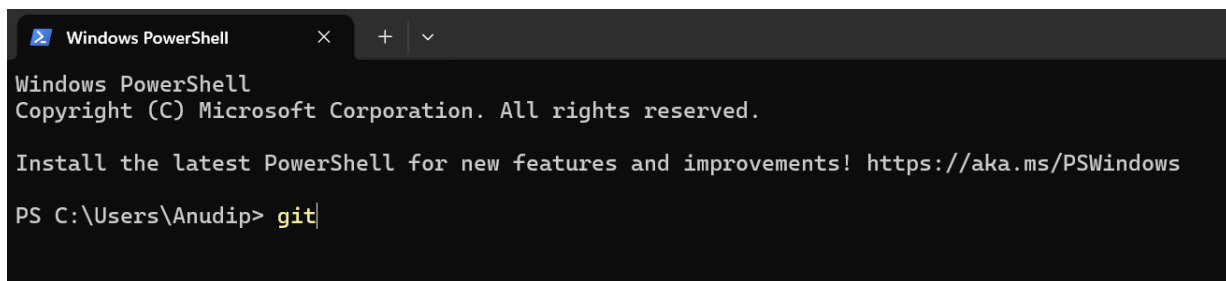
Step 6

Finish installation.

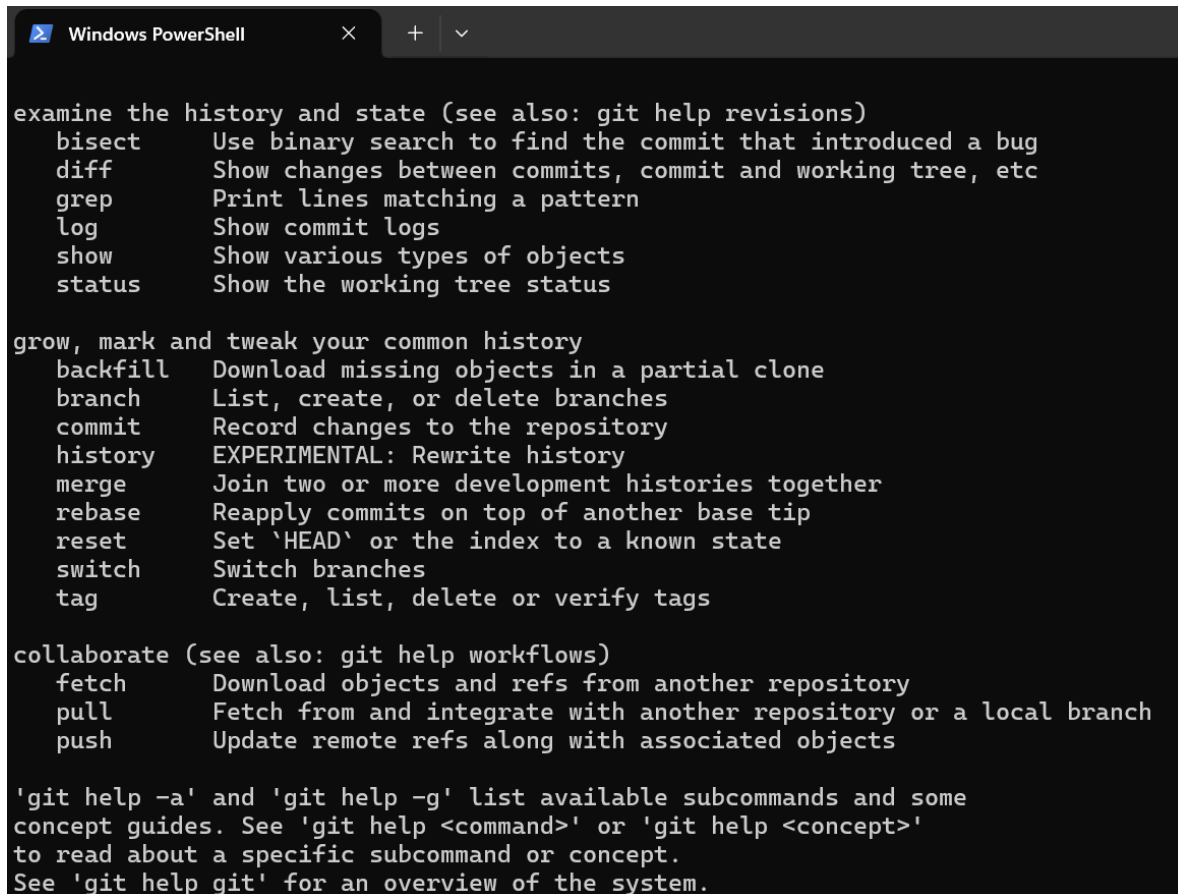


Step 7

Open Command Prompt and run the command git.



It shows all the commands available.



```
Windows PowerShell

examine the history and state (see also: git help revisions)
  bisect      Use binary search to find the commit that introduced a bug
  diff        Show changes between commits, commit and working tree, etc
  grep        Print lines matching a pattern
  log         Show commit logs
  show        Show various types of objects
  status      Show the working tree status

grow, mark and tweak your common history
  backfill    Download missing objects in a partial clone
  branch      List, create, or delete branches
  commit      Record changes to the repository
  history     EXPERIMENTAL: Rewrite history
  merge       Join two or more development histories together
  rebase      Reapply commits on top of another base tip
  reset       Set 'HEAD' or the index to a known state
  switch      Switch branches
  tag         Create, list, delete or verify tags

collaborate (see also: git help workflows)
  fetch       Download objects and refs from another repository
  pull        Fetch from and integrate with another repository or a local branch
  push        Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.
```

macOS Installation

Install Homebrew first:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Install Git: brew install git

Configuring Git

Git needs your username and email.

Open command prompt or Git Bash.

Set Username – Type the following

```
git config --global user.name "Your Name"
```

Set Email

git config --global user.email "youremail@gmail.com"

Verify Configuration

git config --list

Expected Output for git config --list

user.name=John Doe

user.email=johndoe@gmail.com

core.autocrlf=true

credential.helper=manager

init.defaultbranch=main

pull.rebase=false

Explanation of Important Lines

Output	Meaning
user.name=John Doe	Git username configured on the system
user.email=johndoe@gmail.com	Email linked to Git commits
core.autocrlf=true	Automatically manages line endings in Windows
credential.helper=manager	Stores Git credentials securely
init.defaultbranch=main	New repositories use main as default branch
pull.rebase=false	Uses merge strategy during pull operations

If user.name and user.email do not appear, Git has not been configured yet.

Run:

git config --global user.name "Your Name"

git config --global user.email youremail@gmail.com

Then check again using:

git config --list

Creating Your First Git Repository

Step 1: Create Folder

mkdir git-practice

Step 2: Enter Folder

cd git-practice

Step 3: Open in code editor

Code .

Step 4: Go back to bash and create a new markdown file in our folder

touch hello.md

Navigate to vs code we will see the file hello.md

Step 5: Go back to terminal/bash

git status

This will return an error message as git is not initialized.

```
$ git status
fatal: not a git repository (or any of the parent directories): .git
```

Step 6: Initialize Git

To initialize Git, we have to create a new Git repository. This is the first command you run in a new project.

git init

Check status again.

git status

```
On branch main
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello.md

nothing added to commit but untracked files present (use "git add" to track)
```

Step 7: Add File

git add .

This will add all the changes from the working directory in the staging area.

Run git status again.

```
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.md
```

Notice the difference in the output.

Now, in VS code, type:

#I'm learning to use GIT

Save the file.

Run **git status**

```
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hello.md
```

It shows that the file is modified.

Run **git add .**

Let's create another file

touch newfile.js

git status

```
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        newfile.js
```

Step 8: Commit changes

git commit -m "Initial commit"


```
$ git commit -m "Initial commit"
[main (root-commit) cdba41c] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 hello.md
```

This allows us to save our changes.

Standard Workflow

Working Directory

↓

Staging Area

↓

Commit

↓

GitHub

What is a Repository (Repo)?

A repository (or repo) is the main workspace/project folder in GitHub and Git.

It is used to:

- Store project files and code
- Track changes over time using version control
- Maintain project history
- Allow collaboration with multiple people

Simple Definition

A repository is like a smart project folder that remembers:

- what changed,
- who changed it,
- and when it changed.

Why Repositories Matter

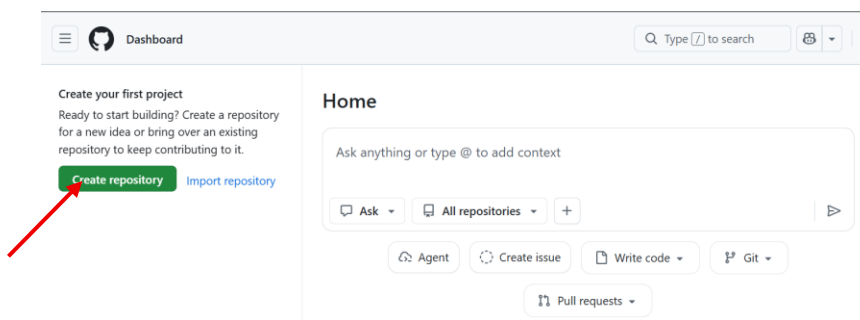
Repositories help developers:

- Work on projects safely
- Collaborate with teams
- Restore previous versions if mistakes happen
- Share open-source projects publicly

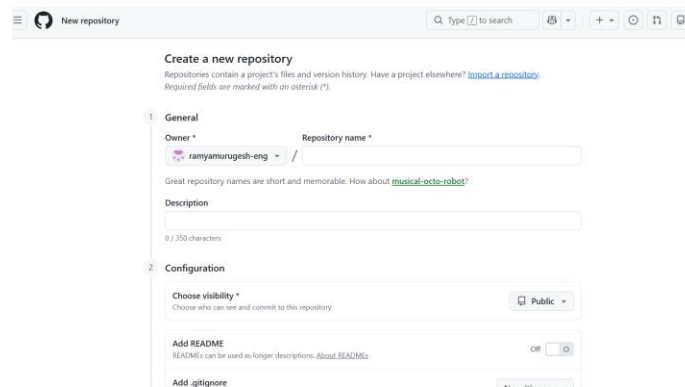
Creating Your First Repository on GitHub

Step 1: Click “New Repository”

After signing into [GitHub](#), click the big green “Create repository” button.



It takes you to this new repository page.



Step 2: Enter Repository Details

Repository Name

Example: **first-repo**

Description

A short explanation about the project.

Example: **My first GitHub repository**

Step 3: Choose Repository Visibility

Public Repository

Anyone on the internet can view it.

Use when:

- sharing projects,
- building portfolio,
- contributing to open source.

Private Repository

Only invited users can access it.

Use when:

- working on confidential projects,
- learning privately,
- storing sensitive work.

Step 4: Initialize with README

What is a README?

A README file explains:

- what the project does,
- why it exists,
- how to use it.

Usually named: README.md

Why it is Important

It acts like the homepage/documentation of the repository.

Step 5: Add a License

Example used: MIT License

Purpose of a License

A license tells others:

- what they are allowed to do with your code,
- what restrictions exist.

Step 6: Click “Create Repository”

GitHub creates the repository instantly.

ramyamuruges-eng

 /

first-repo

✔ first-repo is available.

Great repository names are short and memorable. How about [musical-octo-robot?](#)

Description

My first GitHub repository

26 / 350 characters

Configuration

Choose visibility *
Choose who can see and commit to this repository

Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

On ☒

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#)

MIT License

Create repository

ramyamuruges-eng / first-repo

Type to search

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

first-repo Public

Pin Watch Fork Star

main 1 Branch 0 Tags

Go to file

Add file <> Code

ramyamuruges-eng Initial commit 974327c · now 1 Commit

LICENSE Initial commit now

README.md Initial commit now

README License

first-repo

My first GitHub repository

About
My first GitHub repository
Readme
Activity
0 stars
0 watching
0 forks

Releases
No releases published
[Create a new release](#)

Packages

Understanding Licenses

What is a License?

A license is a legal permission document for your code.

It defines:

- usage rights,
- modification rights,
- sharing rights.

MIT License Features

It is a:

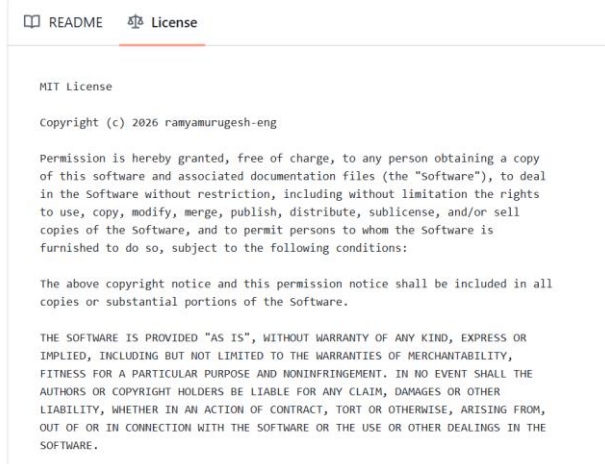
- permissive,
- open-source license.

People can:

- use your code,
- modify it,
- distribute it,
- use it commercially.

Condition:

- They must keep the original license file.



Important Concept: Open Source

Open source means:

- your source code is publicly available,
- others can learn from it,
- improve it,
- or build on top of it.

Useful Resource for Licenses

Choose a License

choosealicense.com

This website helps decide:

- which license fits your project,
- what permissions/restrictions each license has.

Forking a Repository

What is Forking?

Forking means:

- creating your own copy of someone else's repository.

You are NOT starting from scratch.

Why Fork a Repo?

Used mainly in:

- open-source contribution,
- experimentation,
- customization.

Workflow

1. Fork repository

2. Copy goes to your GitHub account
3. Make changes safely
4. Submit changes back using Pull Request

Important Clarification

When someone forks your repository:

- your original repository is NOT automatically changed.

Their changes stay separate unless:

- you approve and merge them.

The Beauty of Open Source

The speakers mention:

“Standing on the shoulders of giants.”

Meaning:

- developers build upon existing work instead of reinventing everything.

What is a .gitignore File?

A `.gitignore` file tells Git:

“Ignore these files/folders. Do not track them.”

Why Use .gitignore?

Some files:

- are unnecessary,
- auto-generated,
- system-specific,
- temporary.

You usually do NOT want them in GitHub.

Example:

```
# Windows system files
Thumbs.db
Desktop.ini
```

macOS Hidden File
.DS_Store

Why Ignore It?

You usually do NOT want it in Git repositories because:

- it is auto-generated,
- unnecessary for the project,
- differs from system to system,
- creates unwanted changes in Git.

Benefits of .gitignore

Keeps repository:

- clean,
- professional,
- lightweight.

Useful Resource for .gitignore

Gitignore Generator

gitignore.io

Generates .gitignore templates automatically for:

- macOS,
- Windows,
- Python,
- Node.js,
- Java,
- VS Code,
- etc.

Common GitHub Repository Tabs

1. Code

Main section containing:

- files,
- folders,
- commits.

2. Issues

Used to:

- report bugs,
- track tasks,
- discuss improvements.

Think of it as: a project task tracker.

3. Pull Requests

Used to:

- submit changes,
- review code,
- merge contributions.

Especially important in:

- teamwork,
- open-source projects.

4. Projects

Used for:

- project management,
- Kanban boards,
- workflow tracking.

Example columns:

- To Do

- In Progress
- Done

5. Actions

Used for automation.

Example:

- automatic testing,
- deployment,
- CI/CD pipelines.

6. Wiki

Used for:

- documentation,
- FAQs,
- guides.

Optional feature.

7. Repository Settings

The **Settings** tab controls:

- permissions,
- security,
- integrations,
- repository features.

Collaborators

Invite people to work on the repository.

Example:

- teammates,
- contributors,
- students.

Security

Controls:

- access permissions,
- repository protection,
- security configurations.

GitHub Actions Automation

Can configure:

- continuous integration (CI),
- continuous deployment (CD).

Best Practices

Always Add:

- README
- Proper repository name
- License
- .gitignore

Keep Repositories Organized

- Use folders
- Use meaningful file names
- Write descriptions clearly

Real-World Workflow Example

Example Project Flow

Create Repo : student-portfolio

Add Files:

index.html

style.css

Commit Changes

Git tracks project history.

Push to GitHub

Upload repository online.

Collaborate

Others can:

- fork,
- clone,
- contribute.

Uploading Files and Folders to GitHub

1. Using the GitHub Website UI

2. Using Git Commands in Terminal

Method 1: Upload Files Using GitHub UI

This is the easiest method for beginners.

Best for:

- small files,
- quick uploads,
- simple repositories.

Step 1: Create a New Repository

Go to [GitHub](#)

Click: New Repository

Step 2: Fill Repository Details

Repository Owner

Select your GitHub account.

Repository Name

Example: insights-storytelling-notes

Description

Short explanation of the project.

Example: Training material covering business insights, chart interpretation, and data storytelling concepts for analytics students.

Visibility

Choose: Public

Check - Add a README file

Step 3

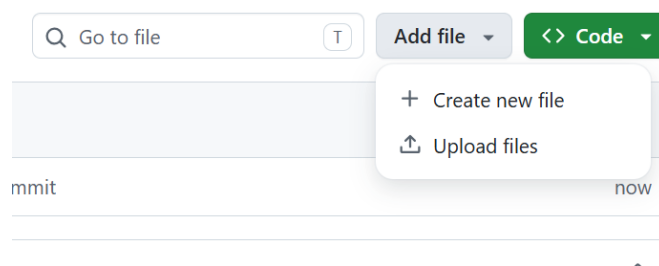
Click: Create Repository

Step 4

Inside Repository

Click the “**Add File**” button

Select: Upload files



Step 5: Choose Files

Click: Choose your files

Select files from your computer.

You can:

- drag and drop,
- or browse manually.

Step 6: Commit Changes

After files upload:

Add a commit message.

Example: Added project files

Then click: Commit changes

What is a Commit?

A commit is:

a saved snapshot of your project.

Each commit stores:

- what changed,
- who changed it,
- when it changed.

Advantages of GitHub UI Upload

Good For

- beginners,
- quick uploads,
- small projects,
- uploading documents/images.

Limitations of GitHub UI Upload

Not ideal for:

- large projects,
- many folders,
- continuous development,
- professional workflows.

For that, terminal/git commands are better.

Method 2: Upload Project Using Terminal (Professional Method)

This is the standard developer workflow.

Best for:

- coding projects,

- multiple folders/files,
- real development work.

Creating an Empty Repository

This time:

- create repository WITHOUT README,
- WITHOUT .gitignore,
- WITHOUT license.

This creates a bare repository.

Why Create an Empty Repository?

Because:

- your project already exists locally,
- GitHub should not create conflicting files.

GitHub Provides Commands

After creating the repository,
GitHub automatically shows commands like:

...or create a new repository on the command line

```
echo "# power-pivot-training-material" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ramyamurugesh-eng/power-pivot-training-material.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/ramyamurugesh-eng/power-pivot-training-material.git
git branch -M main
git push -u origin main
```

Let's follow these steps.

Why GitHub suggests these steps

- `echo "# project-name" >> README.md` → Creates a README file with the project title.
- `git init` → Initializes Git locally.
- `git add README.md` → Stages the README file.
- `git commit -m "first commit"` → Saves the first snapshot.
- `git branch -M main` → Renames the default branch to main (important if Git created master).
- `git remote add origin ...` → Connects your local repo to GitHub.
- `git push -u origin main` → Uploads your code to GitHub.

GitHub shows a generic set of commands for creating a new repository from scratch. If you already have a project locally, you don't need to follow the echo step, and you only rename the branch if it isn't already main.

Local Machine Workflow

Step 1: Open Terminal

Open Git Bash

Step 2: Navigate to Project Folder

Use: `cd folder-name`

Example: **`cd "E:\Anudip\Excel\Power Pivot"`**

`cd` means change directory. Used to move between folders in terminal.

Step 3: Initialize Git

Run: **`git init`**

What Does git init Do?

It converts a normal folder into a Git repository.

Creates hidden: `.git` folder internally

This folder stores:

- commit history,
- branches,

- configuration,
- tracking information.

Step 4: Create README File

Command: **touch README.md**

What is README.md?

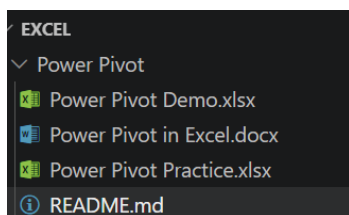
Project documentation file.

Usually contains:

- project name,
- description,
- setup instructions,
- usage guide.

Now, in VS code, you will find the README.md file and type a brief description.

Example: This file describes the data modelling using Power Pivot in Excel



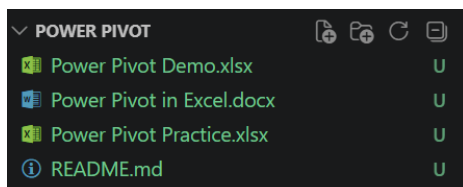
Step 5: Open Project in VS Code

Command: **code .**

What is code .?

- code launches **Visual Studio Code**.
- . means “open the current folder.”

Result: VS Code opens with all your project files visible in the Explorer sidebar.



Step 6: Add Files to Staging Area

Command: **git add .**

What Does git add . Mean?

Adds all files/folders to the staging area.

The dot: . means everything in current folder.

Git Workflow Understanding

Working Directory

Your actual files.

↓

Staging Area

Prepared changes waiting for commit.

↓

Commit

Permanent saved snapshot.

Step 7: Commit files

Command: **git commit -m "initial commit"**

Example : git commit -m "Initial project upload with Power Pivot files"

Understanding This Command

git commit

Creates a snapshot/version.

-m

Adds commit message.

"initial commit"

Describes what changes were saved.

Good Commit Message Examples

Added login page

Fixed navbar bug

Updated README

Step 8: Check Git Status

Command: **git status**.

We can see the files are ready to be pushed to the repository.

```
mac@pebbles:~/Documents/Projects/nextjs$ git status
On branch main
nothing to commit, working tree clean
```

Purpose of git status

Shows:

- modified files,
- staged files,
- untracked files,
- current branch.

Very important debugging command.

Step 9: Rename Branch to Main

Command: **git branch -M main**

What This Does

Renames current branch to: **main**

Modern Git Repositories use main as default branch.

Step 10: Connect Local Repo to GitHub

Command: **git remote add origin <your-repo-URL>**

Example : `git remote add origin https://github.com/username/project.git`

Do not copy the example URL from this document.

Replace <your-repo-URL> with the actual link of your own repository on GitHub.

Understanding This Command

remote

External repository connection.

origin

Default nickname for GitHub repository.

Repository URL

Destination where files will be uploaded.

Step 11: Push Files to GitHub

Command: **git push -u origin main**

Understanding Push Command

push

Uploads commits to GitHub.

-u

Sets upstream tracking.

Future pushes become simpler.

origin

Remote repository name.

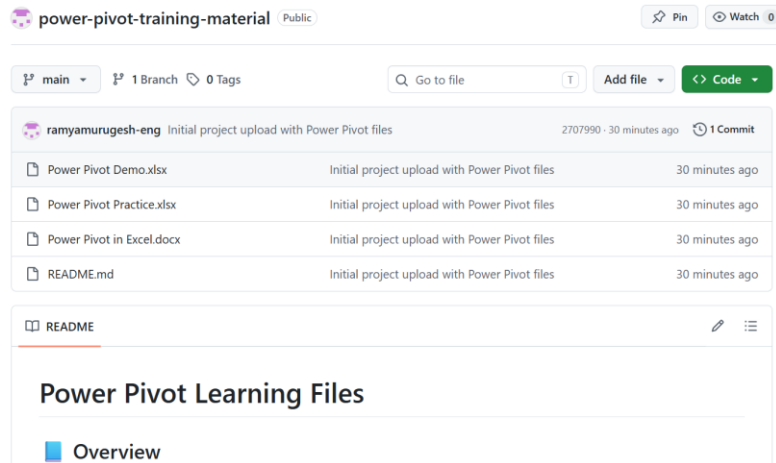
main

Branch name.

Final Result

After refreshing GitHub:

- files appear online,
- repository becomes fully uploaded.



If Git was already initialized earlier,
you DO NOT need: git init again.

You only need:

- remote connection,
- push commands.

Full Command Sequence

cd project-folder

git init

git add .

git commit -m "initial commit"

git branch -M main

git remote add origin REPOSITORY_URL

git push -u origin main

Common Beginner Errors

Error: **Repository Not Found**

Cause:

- wrong repository URL,
- not logged into GitHub.

Error: **git Not Recognized**

Cause:

- Git not installed,
- Git not added to PATH.

Install from:

[Git Downloads](#)

Error: **Permission Denied**

Cause:

- authentication issue,
- incorrect credentials.

Usually fixed using:

- GitHub login,
- personal access token.

Important Git Concepts Learned

Concept	Meaning
Repository	Project storage
Commit	Saved snapshot
Staging Area	Preparation area
Push	Upload to GitHub
Remote	Online repository
Branch	Independent version line

GitHub UI vs Terminal Method

GitHub UI	Terminal Method
Easy	Professional
Beginner friendly	Developer workflow
Small uploads	Large projects
Manual	Efficient
Limited control	Full Git control

The Git Workflow is:

Create Project



Initialize Git



Add Files



Commit Changes



Connect to GitHub



Push to Remote Repository

GitHub stores your project online,
while Git tracks your project history locally.

Introduction to Pull Requests (PR)

A **Pull Request (PR)** is a request to merge changes from one branch into another branch.

Most commonly:

- You make changes in your own branch
- Then request GitHub to merge those changes into the main branch

A PR allows:

- Code review
- Discussion
- Testing before merging
- Collaboration with team members

Simple Real-World Understanding

Imagine:

- main branch = Original project
- Your branch = Your copy where you work safely

You:

1. Create a branch
2. Make changes
3. Push branch to GitHub
4. Create PR
5. Ask team to review
6. Merge into main

Important Terms

Term	Meaning
Branch	Separate workspace for changes
Source Branch	Branch containing your changes
Target Branch	Branch you want to merge into
Pull Request	Proposal to merge changes
Merge	Combining branches
Diff	Difference between files

Where Can PRs Be Created?

You can create PRs using:

- [GitHub Website](#)
- [GitHub Desktop](#)
- [GitHub CLI](#)
- VS Code
- Terminal/Git Bash

We'll use:

- Git Bash
- GitHub website

Creating a Pull Request in GitHub — Step-by-Step

Step 1 — Open Your GitHub Repository

Go to your repository on [GitHub](#)

Example: <https://github.com/username/repository-name>

Step 2 — Copy Repository URL

Click the green: **Code button**

Copy the HTTPS URL.

Example: <https://github.com/username/repository-name.git>

Step 3 — Open Git Bash

Open Git Bash on your system.

Step 4 — Clone the Repository

Use the copied URL to clone the repository.

Command: `git clone <repository-url>`

Example: `git clone https://github.com/username/repository-name.git`

Git will download the repository and create a folder automatically.

Step 5 — Move into the Repository Folder

Command: `cd repository-name`

Example: `cd insights-storytelling-notes`

Step 6 — Create a New Branch

Create and switch to a new branch.

Command: `git checkout -b update-name`

Example Output: Switched to a new branch 'update-name'

Step 7 — Make Changes to the Repository

Open the repository in VS Code or any editor.

Command: `code .`

Step 8 — Create or Edit a File

Create a new file inside the repository.

Example: `practice-notes.txt`

Add sample content to the file and save it.

Step 9 — Stage the Changes

Add the changes to the staging area.

Command: `git add .`

Step 10 — Commit the Changes

Commit the staged changes with a message.

Command: `git commit -m "Added practice notes"`

Example Output: `[update-name ab4a7d9] Added practice notes`

`1 file changed, 1 insertion(+)`

`create mode 100644 Practice-notes.txt`

Step 11 — Verify Repository Status

Check whether the working tree is clean.

Command: `git status`

Expected Output: On branch `update-name`

`nothing to commit, working tree clean`

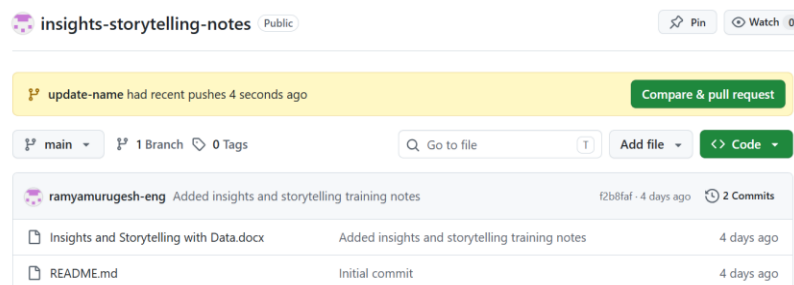
Step 12 — Push the Branch to GitHub

Push the branch to the remote repository.

Command: `git push origin update-name`

Step 13 — Create a Pull Request on GitHub

1. Open the repository on GitHub.



2. Click:

Compare & pull request

3. Add:

- Pull Request title
- Description

4. Click:

Create Pull Request

The Pull Request is now created successfully.

Step 14 — Review the Pull Request

After the Pull Request is created, team members or reviewers can review the proposed changes.

The reviewer can:

- Check the modified files
- Add comments
- Suggest changes
- Approve the Pull Request

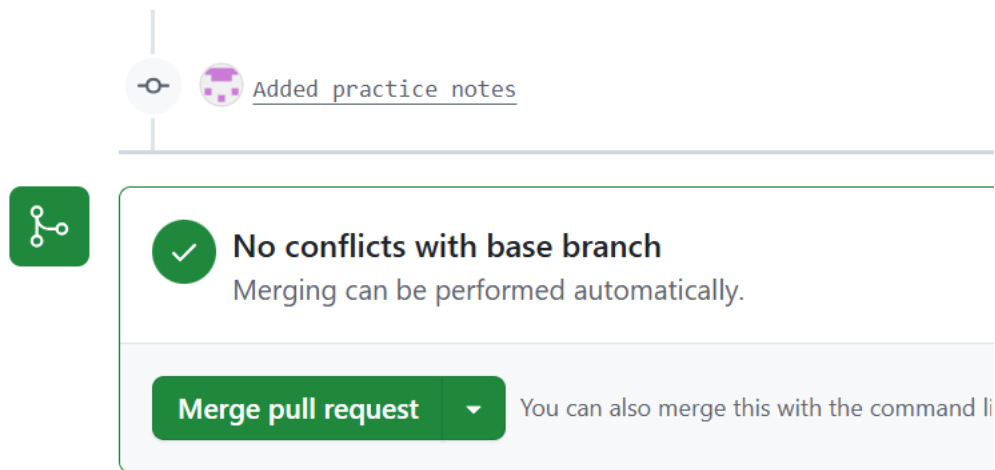
GitHub also shows the differences (“diffs”) between the branches.

Step 15 — Merge the Pull Request

Once the Pull Request is approved:

1. Open the Pull Request on GitHub
2. Click:

Merge Pull Request



3. Then click:

Confirm Merge

The changes from the branch are now merged into the main branch.

Tips for Creating Pull Requests

1. Write Small Pull Requests

Smaller Pull Requests are:

- easier to review
- easier to test
- easier to merge
- less likely to introduce bugs

Instead of making many unrelated changes together, create focused Pull Requests for specific tasks.

2. Review Your Own Pull Request First

Before submitting a Pull Request:

- check the modified files
- review the changes carefully
- test the updates if required
- correct spelling mistakes or errors

This helps reduce review comments and improves code quality.

3. Write Clear Titles and Descriptions

Always provide:

- a meaningful Pull Request title
- a clear description of the changes

A good Pull Request description should include:

- purpose of the changes
- overview of modifications
- additional references if needed

Example:

Purpose:

Practice creating a GitHub Pull Request.

Changes:

- Added Practice-notes.txt
- Added sample notes content

4. Use Meaningful Branch Names

Branch names should describe the work being done.

Examples:

update-readme
add-practice-notes
fix-login-page

Avoid unclear names like:

test1
newbranch
abc

5. Keep Your Repository Updated

Before starting new work:

- pull the latest changes from the main branch
- avoid working on outdated code

Command:

git pull origin main

6. Communicate with Reviewers

If working in a team:

- respond to review comments
- make requested changes
- update the Pull Request when needed

Good communication improves collaboration and project quality.

